



DiabetesRisk

Manual de Instalación



Jared Foster Orduz - Jeferson David Vargas Toca
Andres Felipe Florez Garces - Oscar Enrique Morillo

Universidad de los Andes — Curso MAIA

Septiembre de 2026

1. Introducción

La Este manual describe cómo instalar y poner en marcha, desde cero, los tres componentes de DiabetesRisk (servidor de tracking de MLflow, API de inferencia y tablero Streamlit) usando contenedores Docker. Está escrito para una instancia Ubuntu 24.04 en AWS EC2 (la misma en la que corre el proyecto), pero los pasos aplican igual a cualquier servidor Linux con Docker instalado.

Nota: en la Entrega 2 los tres componentes corrían en entornos virtuales de Python separados. Desde la Entrega Final, el despliegue se hace exclusivamente con Docker; los pasos con entornos virtuales se documentan solo como referencia en el README.md del repositorio.

2. Requisitos previos

- Servidor Linux con Docker Engine y el plugin Docker Compose v2 (se instalan en la sección 3 si no están presentes).
- Acceso por SSH con permisos de sudo.
- Git instalado, para clonar el repositorio.
- Python 3 y DVC instalados en el host, para descargar el dataset ("pip install dvc" o el entorno ya usado en entregas anteriores).
- Puertos abiertos en el grupo de seguridad de AWS (o firewall del servidor): 5000 (MLflow), 8000 (API) y 8501 (tablero).
- Credenciales / sesión AWS activa si el remoto de DVC vive en S3 ("aws sso login" según corresponda).

3. Instalar Docker y el plugin Compose

Si el servidor ya tiene Docker con el subcomando "docker compose" (verificar con el primer comando de abajo), se puede saltar esta sección.

```
docker --version
docker compose version
```

Si el segundo comando falla con "docker: unknown command: docker compose", instalar el repositorio oficial de Docker y el plugin:

```
sudo apt-get update
sudo apt-get install -y ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc

echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc] \
https://download.docker.com/linux/ubuntu $(. /etc/os-release && echo "$VERSION_CODENAME") stable" | \
sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

sudo apt-get update
sudo apt-get install -y docker-compose-plugin
docker compose version
```

4. Clonar el repositorio y obtener el dataset

```
git clone https://github.com/jadfof/Microproyecto_DiabetesRisk.git
cd Microproyecto_DiabetesRisk
python3 -m dvc pull # descarga data/diabetes_binary_health_indicators_BRFSS2015.csv
ls -lh data/ # confirmar que el .csv quedó ahí
```

El dataset se monta como volumen de solo lectura dentro de los contenedores "trainer" y "dashboard"; no se incluye en las imágenes de Docker. Por eso debe existir en el host antes del paso 6.

5. Estructura relevante del repositorio

Carpeta / archivo	Contenido
mlflow/train.py	Script de entrenamiento de los 5 modelos y registro en MLflow.
mlflow/Dockerfile	Imagen usada tanto por el servidor de MLflow como por el job de entrenamiento.
api/main.py, api/Dockerfile	API FastAPI que sirve el modelo empaquetado.
dashboard/app.py, dashboard/Dockerfile	Tablero Streamlit que consume la API.
docker/docker-compose.yml	Orquestación de los 4 servicios (mlflow, trainer, api, dashboard).
data/	Dataset versionado con DVC (no se sube directamente a Git).

6. Puesta en marcha paso a paso

6.1 Levantar el servidor de MLflow

```
cd ~/Microproyecto_DiabetesRisk/docker
docker compose up -d --build mlflow
docker compose ps      # esperar a que aparezca "healthy"
```

Verificar en el navegador: <http://<IP-DEL-SERVIDOR>:5000> — debe cargar la interfaz de MLflow.

6.2 Entrenar los modelos

El entrenamiento es un job de un solo uso (no queda corriendo en segundo plano), por eso usa el "profile" train y no arranca con un "docker compose up" normal:

```
docker compose --profile train run --rm trainer
```

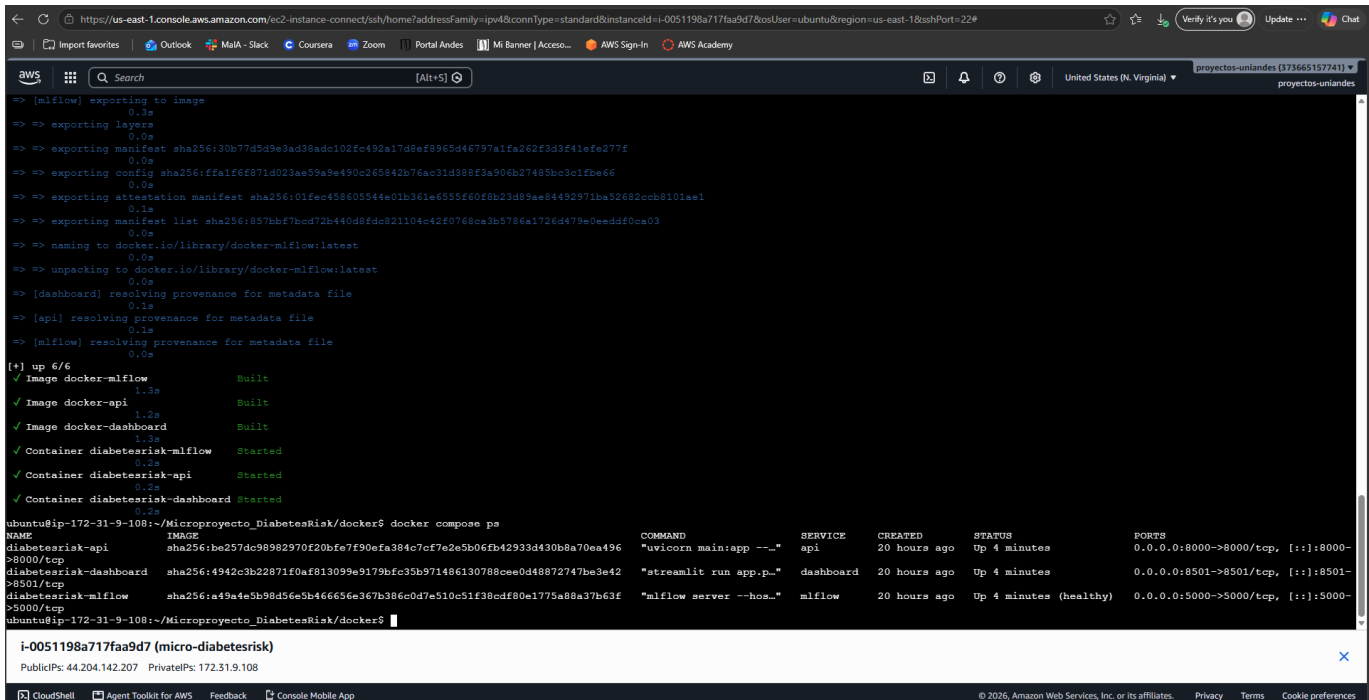
Esto entrena los 5 modelos, registra cada corrida en MLflow y deja el artefacto ganador ("model.pkl" + "metrics.json") en un volumen compartido que la API leerá en el siguiente paso. Debe terminar con una línea similar a:

```
Modelo guardado en mlflow/model.pkl (modelo seleccionado: random_forest_balanced)
```

6.3 Levantar la API y el tablero

```
docker compose up -d --build api dashboard
docker compose ps
```

Deben quedar 3 contenedores en estado "Up": diabetesrisk-mlflow, diabetesrisk-api y diabetesrisk-dashboard.



```

=> [mlflow] exporting to image
0.3s
=> => exporting layers
0.0s
=> => exporting manifest sha256:30b77d5d9e3ad38adc102fc492a17d8ef8965d46797a1fa262f3d3f41efe277f
0.0s
=> => exporting config sha256:ffa1f6f871d023ae59a9e490c265942b76ac31d388f3a906b27485bc3c1fb66
0.0s
=> => exporting attestation manifest sha256:01fec458605544e01b361e655f60f8b23d89ae84492971ba52682ecb8101ae1
0.1s
=> => exporting manifest list sha256:857bbf7bcd72b440d8fde821104e42f0768ca3b5786a1726d479e0eddff0ca03
0.0s
=> => naming to docker.io/library/docker-mlflow:latest
0.0s
=> => unpacking to docker.io/library/docker-mlflow:latest
0.0s
[dashboard] resolving provenance for metadata file
0.1s
=> [api] resolving provenance for metadata file
0.1s
=> [mlflow] resolving provenance for metadata file
0.0s

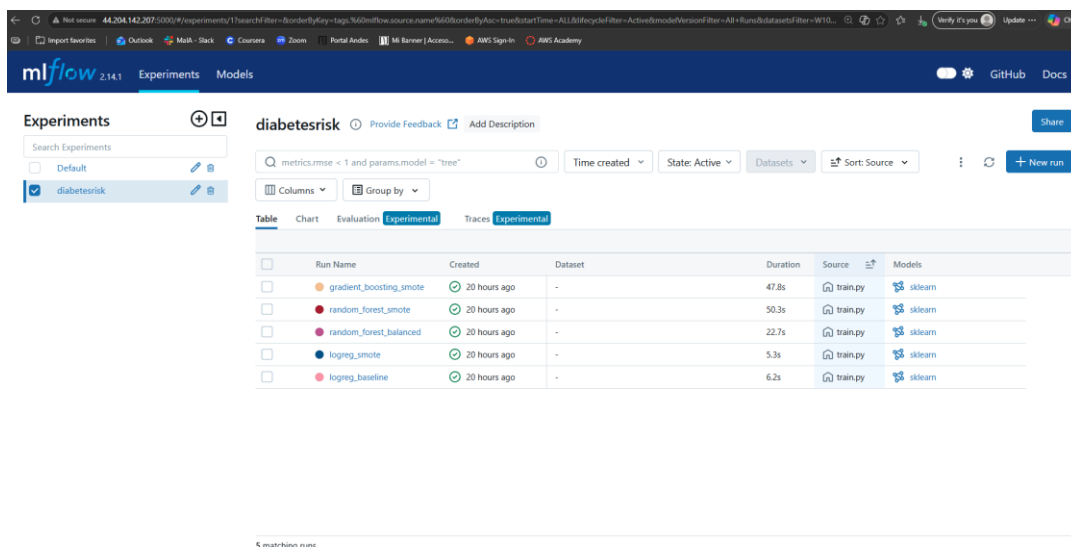
[+] up 6/6
✔ Image docker-mlflow      Built
✔ Image docker-api         Built
✔ Image docker-dashboard   Built
✔ Container diabetesrisk-mlflow Started
✔ Container diabetesrisk-api Started
✔ Container diabetesrisk-dashboard Started

ubuntu@ip-172-31-9-108:~/Microproyecto_DiabetesRisk/docker$ docker compose ps
NAME                IMAGE                                COMMAND                  SERVICE    CREATED   STATUS    PORTS
diabetesrisk-api     sha256:b257dc98982970f20bfe7f90efa38467cf7e2e5b06fb42933d430b8a70ea496  "uvicorn main:app --"    api         20 hours ago Up 4 minutes 0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp
diabetesrisk-dashboard sha256:4942c3b22871f0af813099e9179bfc35b971486130788cee0d48872747be3e42  "streamlit run app.py"  dashboard  20 hours ago Up 4 minutes 0.0.0.0:8501->8501/tcp, [::]:8501->8501/tcp
diabetesrisk-mlflow  sha256:a494fe5b98d56e5b46665e367b386c0d7e510c51f38cdf80e1775a88a37b63f  "mlflow server --hos"   mlflow     20 hours ago Up 4 minutes (healthy) 0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp
  
```

Figura 1. Salida de "docker compose ps" con los 3 servicios levantados.

7. Verificación funcional

- **MLflow:** <http://<IP-DEL-SERVIDOR>:5000> → deben verse las 5 corridas del experimento "diabetesrisk".
- **API:** <http://<IP-DEL-SERVIDOR>:8000/docs> → probar "GET /health" y "POST /predict" desde la interfaz interactiva.
- **Tablero:** <http://<IP-DEL-SERVIDOR>:8501> → completar el formulario y confirmar que aparece un resultado de riesgo.



Run Name	Created	Dataset	Duration	Source	Models
gradient_boosting_smote	20 hours ago	-	47.8s	train.py	sklearn
random_forest_smote	20 hours ago	-	50.3s	train.py	sklearn
random_forest_balanced	20 hours ago	-	22.7s	train.py	sklearn
logreg_smote	20 hours ago	-	5.3s	train.py	sklearn
logreg_baseline	20 hours ago	-	6.2s	train.py	sklearn

Figura 2. Experimento "diabetesrisk" en MLflow, con las 5 corridas generadas por el contenedor "trainer".

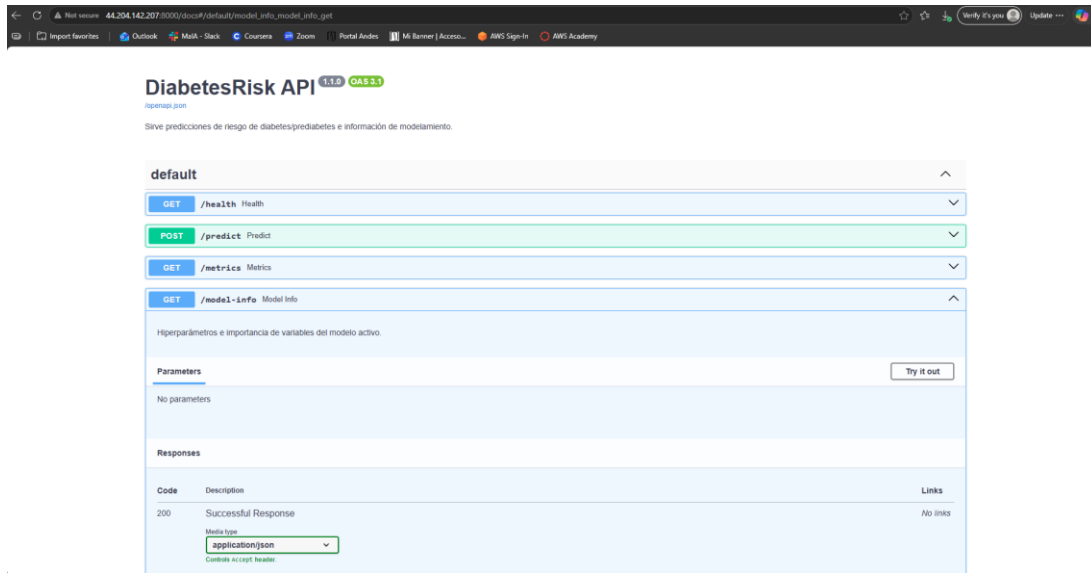


Figura 3. Documentación interactiva de la API ("/docs"), endpoint "/model-info".

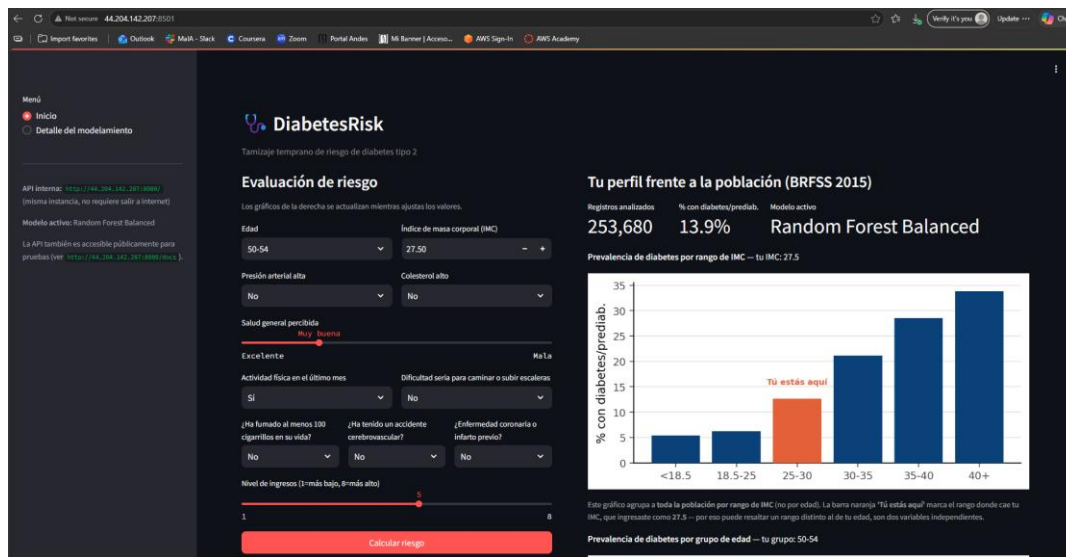


Figura 4. Dashboard con vista de Inicio, parámetros del lado izquierdo y indicadores del lado derecho.

8. Operación diaria

8.1 Ver logs de un servicio

```
docker compose logs -f api
docker compose logs -f dashboard
docker compose logs -f mlflow
docker compose --profile train logs trainer
```

8.2 Reentrenar los modelos

```
docker compose --profile train run --rm trainer
docker compose restart api # para que la API tome el model.pkl más reciente
```

8.3 Actualizar el código (tras un "git pull")

```
git pull
docker compose up -d --build mlflow api dashboard
```

9. Apagar el sistema sin perder información

Para detener los contenedores conservando el modelo entrenado y el historial de MLflow (recomendado al terminar de trabajar, y obligatorio antes de entregar el proyecto según las indicaciones del curso):

```
docker compose stop
```

No usar "docker compose down -v": el flag "-v" borra los volúmenes "mlflow-data" y "model-artifacts", eliminando el historial de MLflow y el modelo entrenado.

10. Solución de problemas comunes

Síntoma	Causa probable	Solución
"docker: unknown command: docker compose"	Solo está instalado Docker Engine, no el plugin Compose v2.	Seguir la sección 3 de este manual (repositorio oficial de Docker + docker-compose-plugin).
El puerto 5000/8000/8501 ya está en uso	Quedó un proceso previo corriendo (por ejemplo, de la versión con entornos virtuales).	Ubicarlo con "ps aux grep -E 'mlflow uvicorn streamlit'" y matarlo con "kill <PID>" antes de levantar los contenedores.
La API responde 503 "Modelo no encontrado"	No se ha corrido el contenedor "trainer" todavía, o se corrió antes de crear el volumen compartido.	Ejecutar "docker compose --profile train run --rm trainer" y luego "docker compose restart api".
"dvc pull" falla por archivos sin guardar	Quedó un "model.pkl" u otro archivo modificado localmente que DVC no quiere sobrescribir.	No es necesario para Docker (el dataset .csv puede ya estar presente); si hace falta forzar, usar "dvc pull --force" con cuidado.
El tablero no puede conectarse a la API	La API no ha terminado de iniciar, o el contenedor "api" no está corriendo.	Revisar "docker compose ps" y "docker compose logs api"; reiniciar con "docker compose up -d --build api".

11. Desinstalación / limpieza completa

Para eliminar contenedores, imágenes y volúmenes del proyecto (borra también el modelo entrenado y el historial de MLflow):

```
docker compose down -v --rmi local
```